

# After the Firestorm

## Rebuilding Technical Organizations from Crisis to Confidence

*Firestorms are survivable, but recovery requires a fundamentally different approach than what created the crisis.*

You already know things are bad. You didn't need a quiz to tell you that. The quiz probably confirmed something you've been feeling for a while. This is multiple fires feeding each other, and the approaches that got the organization to this point are not going to get it out.

This paper is not going to sugarcoat the situation. If you scored in the firestorm range, your organization is dealing with compounding problems that have been building for years. Organizations come back from this. The ones that do share a common trait. They stop trying to fix the symptoms and start making the hard decisions about what to save, what to let go, and how to rebuild trust with the people who've been living in the fire.

---

## **Anatomy of a Firestorm**

Firestorms don't start overnight. They build. One problem feeds another, which feeds another, until the whole system is self-sustaining in the worst possible way.

Technical debt drives turnover. Your best engineers don't want to spend their careers fighting fires and maintaining systems nobody will let them fix. They leave. Turnover concentrates knowledge. The people who leave take context with them, and the people who remain inherit systems they don't fully understand. Concentrated knowledge slows releases. Every change requires the one remaining person who knows how the system works, and that person is already overloaded. Slow releases frustrate customers. Features are late. Bugs take too long to fix. Escalations increase. Customer frustration drives reactive decision-making. Leadership throws resources at the most visible fires. They approve custom deals to retain revenue. They hire in a panic. Each reactive decision adds another layer of complexity to a system that was already collapsing under its own weight.

The cycle continues.

The magic wand question on the quiz hit different for you. You didn't think about polishing the edges or fixing a specific system. You thought about starting over. That impulse is understandable, and it tells you something important about where you are. "Start over" is a feeling, and this paper is about turning it into an actual path forward.

---

## The Turnover-Knowledge Spiral

In a firestorm, turnover is not just a staffing problem. It's the engine that keeps everything else on fire.

Good engineers have options. When they're stuck in an environment where they spend most of their time on incident response, where the systems they work on are fragile and the processes around them are broken, where they've raised the same problems repeatedly and nothing has changed, they start looking. They don't always announce it. They just quietly update their resume, take a few calls, and one day they give notice.

When they leave, they take more than their labor. They take the context that lived in their head. The workarounds they maintained. The relationships with the vendor whose system integrates with yours. The memory of why that configuration is set the way it is. The institutional history that explains decisions that otherwise look irrational.

The remaining team members inherit all of that responsibility without inheriting any of that knowledge. They're now doing more work with less understanding of the systems they're working on. The environment gets worse. The next best person starts looking. The spiral tightens.

Here is the part that leadership often misses. By the time someone gives notice, the decision to leave was made months ago. The thing that pushed them over the edge was not the last incident. It was the accumulation of every incident before it, every ignored recommendation, every "we'll get to that next quarter" that never materialized. By the time you see the resignation, it's far too late to address the cause.



**Stopping the spiral requires addressing the environment, not just replacing the people. If you hire into the same conditions that drove the last person out, the new person will eventually reach the same conclusion. That doesn't solve the problem; it resets the clock on it.**

---

## The Revenue Discipline Problem

Firestorm organizations almost always have a revenue discipline problem. It manifests as a portfolio of custom commitments, one-off implementations, and bespoke features that were each individually justified by a deal but collectively represent an enormous, ongoing cost that nobody accounts for.

The pattern is predictable. A customer offers a significant deal contingent on custom work. Sales is excited. Leadership approves it. The team builds it. The contract is signed. Revenue is recognized.

Then time passes. The customer downsizes their commitment, or leaves entirely, or simply stops using the custom feature. The code remains. It sits in the codebase, untouched, slowly drifting out of compatibility with the rest of the platform. Nobody fully understands it because the people who built it have moved on. It will sit there quietly until it breaks, at which point it becomes a crisis that pulls teams off the work they should be doing.

Multiply that by five custom deals, or ten, or twenty, and you have an organization where a meaningful percentage of the engineering team's capacity is consumed by maintaining things that generate no current revenue and serve no current customer. The developers have to sort through code paths that exist for one client who left three years ago. Every new feature has to account for edge cases that only exist because of bespoke implementations nobody remembers agreeing to.

The revenue from those original deals was real. It was a one-time capture. The maintenance cost is perpetual, and it was never factored into the original calculation, because nobody asked "what does this cost us in year five?" or "what happens when they don't renew?"

Then there are acquisitions. An acquisition is, in many ways, the largest one-customer feature an organization can take on. You acquire a company or a product, announce the integration, and then the actual work of absorbing it begins. Except it often doesn't. The integration stalls. The acquired system runs on different infrastructure, different frameworks, different conventions. The original team that built it leaves. Ten years after the acquisition announcement, the legacy code from the acquired product is still sitting in its own silo, maintained by whoever happens to be closest to it, running on infrastructure nobody else

touches. It was supposed to be temporary. It became permanent through neglect. Now it's another fire that feeds the rest.

Unwinding this in a firestorm is painful. It means sunseting features that someone, somewhere, might still be using. It means having conversations with customers about what will and won't be supported going forward. It means acknowledging that decisions made years ago, decisions that looked smart at the time, created obligations the organization can no longer afford to carry.

---

## **Triage: What to Save, What to Sunset, What to Rebuild**

In a firestorm, the most important skill is triage, and triage means being clear-headed about what is and is not worth saving.

### **What to save:**

Not everything is broken. Even in a firestorm, there are systems that work, teams that function, and processes that hold up under pressure. Identify these clearly and protect them. They're your foundation. Every recovery needs something stable to build from, and the mistake is assuming everything needs to be torn down when some of it just needs to be left alone.

### **What to sunset:**

Look for the things the organization is carrying out of inertia. The custom implementations nobody uses. The internal tools that three people rely on but cost a full-time engineer to maintain. The integrations with vendors whose contracts ended two years ago. The features that were built for a market position the company no longer occupies.

Sunseting is discipline. Every system you maintain that doesn't serve the current business is capacity you're not spending on the things that do. In a firestorm, you cannot afford the luxury of carrying dead weight.

### **What to rebuild:**

Some things are too broken to fix incrementally. The question is whether a rebuild is genuinely the right call or whether it's an emotional response to the pain of the current state.

Before committing to a rebuild, ask these questions. Can you clearly articulate what the rebuilt system needs to do that the current system cannot? Do you have the people and the stability to execute a rebuild without the new system becoming its own crisis? Can the business survive the timeline a rebuild requires, which is always longer than the initial estimate? If the answer to any of these is no, a rebuild is not the right move yet. Stabilize first. You can revisit the rebuild conversation when you're operating from a position of clarity rather than desperation.

---

## **Stabilize, Then Modernize**

The instinct in a firestorm is to jump straight to the big solution. Rewrite the platform. Migrate to the cloud. Adopt microservices. Replace the entire stack. These are dramatic moves that feel proportional to the scale of the crisis.

They also almost always fail when attempted from inside a firestorm.

A rewrite is a multi-year commitment that requires sustained focus, clear requirements, and a stable team. A firestorm organization has none of those things. What typically happens is the rewrite starts with energy and executive support, runs into the reality of maintaining the existing system while simultaneously building the new one, and eventually stalls when a more urgent fire demands attention. Six months later, you have two systems, neither of which works well, and a team that's more demoralized than before because they can see the half-built future but can't get to it.

Stabilize first. Get to a state where the existing systems are not actively failing. Address the knowledge concentration. Fix the deployment pipeline. Establish monitoring so you know what's actually happening. Build the muscle of shipping small changes safely and frequently.

Then modernize from a position of strength. Replace systems one at a time. Migrate incrementally. Make each step reversible. This is slower than a big-bang rewrite, and it is dramatically more likely to succeed.

There is a trap on this side too. Incremental migration only works if you actually follow through. If you start migrating incrementally but keep deferring the next step in favor of shiny new features, you end up in the same place that brought you to the firestorm. You'll have a half-migrated system that's worse than what you started with, because now you're

maintaining two architectures instead of one. Discipline and focus are the difference between incremental progress and incremental neglect. That discipline is harder on leadership than on the team, because it means doing the unglamorous work of negotiating priorities, protecting the migration time, and saying no to things that are more exciting but less important.

---

## **Rebuilding Team Confidence**

The human side of a firestorm is the part that's easiest to overlook and hardest to recover from.

In a brush fire, the team quietly loses the habit of improvement. In a smolder, they stop trusting that leadership will act. In a crown fire, they stop functioning as a team and fragment into individuals managing their own emergencies. In a firestorm, the team is dissolving.

Your best people have already left or are actively planning to. The ones who remain are exhausted and, in many cases, have checked out emotionally even if they still show up every day. They've been firefighting, absorbing the knowledge of departed colleagues, working around broken systems, and watching their recommendations get ignored for months or years. The idea that things could actually get better feels less like hope and more like a story leadership tells itself. The team has no reason to believe that this time will be different, because every previous "this time" wasn't.

What you're looking at is a group of people who have lost the belief that the organization can solve its own problems. Not just that leadership won't prioritize it, but that the problems might be beyond solving. That's a fundamentally different place than a smolder or a crown fire. In those states, the team still believes improvement is possible if the organization chose to invest. In a firestorm, that belief is gone. Rebuilding it requires more than promises and plans. It requires visible, irreversible action.

You cannot rebuild team confidence with words. You rebuild it with action. Specifically, you rebuild it with visible, concrete changes that demonstrate the organization is serious about addressing the problems the team has been raising.

Cancel a project to make room for stabilization work. Not quietly. Publicly. Explain why.

Fix one of the things the team has been asking to fix for a year. The specific thing matters less than the signal it sends.

Protect the team from new commitments during the recovery period. Every new feature promise made while the team is stabilizing is a message that the crisis is not actually a priority.

If your credibility as a leader has been damaged by too many unfulfilled promises, acknowledge it directly. "I know we've said we'd address this before and didn't. Here's what's different this time, and here's how you'll be able to see the progress." Then deliver. Words without follow-through will make things worse.

It's called an executive decision for a reason. Live up to that. If the situation calls for hard choices about what to defer, what to sunset, and what to fund, make those choices and own them visibly. Don't push the tradeoff decisions down to the team and then hold them accountable for the outcomes. The team needs to see leadership making real sacrifices of its own.

---

## The 6-Month Transformation Pattern

Six months will not fix everything. Six months of focused, disciplined work can get a firestorm organization from "we're drowning" to "we have a plan and we're executing it." That shift, from reactive to intentional, is what makes the difference between an organization that recovers and one that slowly declines.

### **Months 1 and 2: Assess, triage, and stop the bleeding.**

Map the real state of the organization. Identify the critical failure points, the load-bearing people, and the compounding problems. Decide what to stabilize, what to let burn, and what to let go entirely. Begin addressing the top one or two issues that are either affecting customers directly or blocking every other improvement.

This is also when you address the turnover risk. Talk to the people who are holding things together. Find out what they need to stay. Not pizza parties and motivational emails. Real changes to their working conditions. If you can't deliver those changes, be upfront about the timeline for when you can.



### **Months 3 and 4: Stabilize the critical path and begin knowledge transfer.**

With the most urgent fires contained, start building redundancy into the critical systems. Cross-train the load-bearing people. Document the undocumented organizational knowledge. Begin unwinding the most costly one-customer commitments.

This is the phase where the team starts to feel the difference. Incidents become less frequent. Releases become less terrifying. The team begins to trust that things are actually improving, not just being talked about.

### **Months 5 and 6: Build the foundation for sustained improvement.**

Establish the practices that prevent the next firestorm. A maintenance lane for ongoing debt reduction. Monitoring and alerting that catches problems before customers do. A deployment process that the team trusts. A decision framework for evaluating custom deals that accounts for long-term maintenance cost.

By month six, the goal is an organization that can plainly describe its current state, has a prioritized plan for continued improvement, is not losing people to burnout or frustration, and is shipping with confidence rather than anxiety. There will still be significant work ahead. The firestorm will be out, and the rebuilding will be underway.

#### **Now what**

You have a read on how serious this is. The next step is turning it into a plan for the one thing most in your way. As you read, mark this up: what resonated, what didn't fit, and what you're still unsure about. Then write down the three questions you still have.

---

---

---

**Bring your question to TPM office hours →**

Not ready for that? [Ask us a question for free.](#) We answer the ones that help the most people, on the blog and on socials.

## **About Understory Collaborative**

Understory Collaborative is a team of seasoned technologists who specialize in the work that happens beneath the surface. We're smokejumpers. We've walked into firestorms before and helped organizations find a way through.

We assess before we prescribe. We've seen these patterns before. We plan to leave your organization stronger than we found it. We tell you what you need to hear, even when it's hard.

If anything in this paper sounded familiar, we should talk. Reach out at [contact@understorycollab.com](mailto:contact@understorycollab.com) or visit [understorycollab.com/contact](https://understorycollab.com/contact).